

Compliance & Traceability Matrix

Related: [System Design](#) | [SRS](#) | [BRD](#)

1. Requirements-to-Implementation Traceability

This matrix maps every Business Requirement (BR) and User Story (US) from the [BRD](#) to its concrete implementation artifacts, ensuring full coverage.

1.1 Authentication & Account Management

Req ID	Requirement	Backend Artifact	Frontend Artifact	DB Collection
US-AUTH-01	User login	<code>routes/auth.py::login</code>	<code>LoginPage.js</code>	users, sessions
US-AUTH-02	First-login password change	<code>routes/auth.py::change_password</code>	<code>ForcePasswordChange.js</code>	users
US-AUTH-03	30-min session timeout	<code>auth/security.py::create_access_token</code>	<code>AuthContext.js</code> (401 handler)	sessions
US-AUTH-04	Secure logout	<code>routes/auth.py::logout</code>	<code>AuthContext.js::logout</code>	sessions
US-AUTH-05	Admin password reset	<code>routes/users.py::admin_reset_password</code>	<code>UserManagement.js</code>	users
US-AUTH-06	Self-service password change	<code>routes/auth.py::change_password</code> (verifies current password, not first-login)	<code>ProfilePage.js</code> (change password section)	users
US-AUTH-07	Forgot password	<code>routes/auth.py::forgot_password</code> (notifies admins, prevents enumeration)	<code>ForgotPassword.jsx</code> , <code>LoginPage.js</code>	notifications

1.2 Vehicle Management

Req ID	Requirement	Backend Artifact	Frontend Artifact	DB Collection	Test Coverage
US-VEH-01	Register vehicle	<code>routes/vehicles.py::create_vehicle</code> (with plate uniqueness check, length validation 2-15 chars)	<code>VehiclesPage.js</code>	vehicles	TC-VEH-01, TC-VEH-07, TC-VEH-08
US-VEH-02	Manage multiple vehicles	<code>routes/vehicles.py::get_vehicles, delete_vehicle</code>	<code>VehiclesPage.js</code>	vehicles	TC-VEH-02
US-VEH-03	Dashboard vehicle display	—	<code>UserDashboard.js</code>	vehicles	TC-VEH-03

1.3 Parking Reservation

Req ID	Requirement	Backend Artifact	Frontend Artifact	DB Col
US-RES-01	Book parking slot	<code>routes/reservations.py::create_reservation</code>	<code>BookingPage.js</code>	reserva
US-RES-02	Multi-date booking	<code>routes/reservations.py::create_reservation</code> (loop)	<code>BookingPage.js</code> (date picker)	reserva
US-RES-03	View available slots	<code>routes/buildings.py::get_available_slots</code>	<code>BookingPage.js</code> (slot grid)	parking reserva
US-RES-04	QR code	<code>routes/reservations.py::get_reservation_qr</code>	<code>ReservationsPage.js</code>	reserva
US-RES-05	Cancel reservation	<code>routes/reservations.py::cancel_reservation</code>	<code>ReservationsPage.js</code> , <code>UserDashboard.js</code>	reserva
US-RES-06	Active/History tabs	—	<code>UserDashboard.js</code>	reserva
US-RES-07	Default booking times	<code>routes/parking_config.py</code>	<code>BookingPage.js</code>	parking users

Req ID	Requirement	Backend Artifact	Frontend Artifact	DB Col
US-RES-08	Profile booking preferences	<code>routes/auth.py::update_booking_preferences</code>	<code>ProfilePage.js</code>	users
US-RES-09	Zone building count	<code>routes/zones.py::get_user_building_assignments</code>	<code>UserDashboard.js</code>	zones
US-RES-10	Notifications	<code>routes/notifications.py</code>	<code>NotificationBell.js</code>	notification
US-RES-11	Hourly slot timeline	<code>routes/waitlist.py::get_floor_timeline</code>	<code>BookingPage.js</code> (timeline grid)	parking reservation
US-RES-12	Waitlist	<code>routes/waitlist.py</code> (join, leave, status, count)	<code>BookingPage.js</code> (waitlist button)	waitlist, notification

1.4 Zone & Building Rules

Req ID	Requirement	Backend Artifact	Frontend Artifact	DB Collection
US-ZONE-01	Main building booking	<code>routes/reservations.py</code> (no extra validation)	<code>BookingPage.js</code>	users, reservations
US-ZONE-02	Zone building access	<code>routes/zones.py::get_user_building_assignments</code>	<code>BookingPage.js</code>	zones
US-ZONE-03	External building rules	<code>routes/reservations.py</code> (single-day + reason)	<code>BookingPage.js</code>	zones, reservations

1.5 Admin — User Management

Req ID	Requirement	Backend Artifact	Frontend Artifact	DB Collection
US-ADM-01	Create user	<code>routes/users.py::create_user</code>	<code>UserManagement.js</code>	users
US-ADM-02	Bulk upload	<code>routes/users.py::bulk_upload_users</code>	<code>UserManagement.js</code>	users, zones
US-ADM-03	Block/unblock	<code>routes/users.py::block_user, unblock_user</code>	<code>UserManagement.js</code>	users
US-ADM-04	Main building assignment	<code>routes/users.py::assign_main_building</code>	<code>UserManagement.js</code>	users

Req ID	Requirement	Backend Artifact	Frontend Artifact	DB Collection
US-ADM-05	VIP/Group Head tags	<code>routes/users.py::update_user_tags</code>	<code>UserManagement.js</code>	users
US-ADM-06	Default booking times	<code>routes/users.py::update_user</code>	<code>UserManagement.js</code>	users
US-ADM-07	Advanced filters	—	<code>UserManagement.js</code>	—
US-ADM-08	Bulk modify	<code>routes/users.py::bulk_set_main_building, bulk_assign_zone</code>	<code>UserManagement.js</code>	users, zones
US-ADM-09	Template download	<code>routes/templates.py::download_users_template</code>	<code>UserManagement.js</code>	—
US-ADM-10	Audit timestamps	<code>routes/users.py</code> (created_at/updated_at)	<code>UserManagement.js</code>	users
US-ADM-11	Waitlist configuration	<code>routes/parking_config.py</code> (waitlist_enabled, window)	<code>ParkingConfig.js</code>	parking_conf
US-ADM-12	Job Family field	<code>routes/users.py</code> (CRUD + bulk upload)	<code>UserManagement.js</code>	users

1.6 Admin — Building & Slot Management

Req ID	Requirement	Backend Artifact	Frontend Artifact	DB Collection
US-BLD-01	Create building	<code>routes/buildings.py::create_building</code>	<code>BuildingManagement.js</code>	buildings
US-BLD-02	Add floors with layout	<code>routes/buildings.py::add_floor, upload_floor_layout</code>	<code>BuildingManagement.js</code>	floors
US-BLD-03	Add slots	<code>routes/buildings.py::add_slots_to_floor</code>	<code>BuildingManagement.js</code>	parking_slots
US-BLD-04	Block/unblock slots	<code>routes/buildings.py::update_slot_status</code>	<code>BuildingManagement.js</code>	parking_slots
US-BLD-05	Delete slots	<code>routes/buildings.py::delete_slot</code>	<code>BuildingManagement.js</code>	parking_slots
			<code>BookingPage.js</code>	

Req ID	Requirement	Backend Artifact	Frontend Artifact	DB Collection
US-BLD-06	Available slots	<code>routes/buildings.py::get_available_slots</code>		parking_slots reservations

1.7 Admin — Zone Management

Req ID	Requirement	Backend Artifact	Frontend Artifact	DB Collection	Test Coverage	Status
US-ZN-01	Create zone	<code>routes/zones.py::create_zone</code>	<code>ZoneManagement.js</code>	zones	TC-ZN-01	Implem
US-ZN-02	Assign users	<code>routes/zones.py::update_zone</code>	<code>ZoneManagement.js</code>	zones	TC-ZN-02	Implem
US-ZN-03	Manage buildings	<code>routes/zones.py::update_zone</code>	<code>ZoneManagement.js</code>	zones	TC-ZN-03	Implem

1.8 Admin — Parking Configuration

Req ID	Requirement	Backend Artifact	Frontend Artifact	DB Collection
US-CFG-01	Default parking hours	<code>routes/parking_config.py::save_parking_config</code>	<code>ParkingConfig.js</code>	parking_conf
US-CFG-02	Booking window	<code>routes/parking_config.py</code>	<code>ParkingConfig.js</code>	parking_conf
US-CFG-03	Release time	<code>routes/parking_config.py</code>	<code>ParkingConfig.js</code>	parking_conf
US-CFG-04	Auto no-show release	<code>services/background.py::auto_mark_no_shows</code>	<code>ParkingConfig.js</code>	parking_conf reservations
US-CFG-05	Main building exclusivity	<code>routes/parking_config.py</code> (<code>main_building_exclusive</code> field); <code>routes/reservations.py::create_reservation</code> (exclusivity check)	<code>ParkingConfig.js</code> (toggle)	parking_conf

1.9 Admin — Reports & Analytics

Req ID	Requirement	Backend Artifact	Frontend Artifact	DB Collection
US-RPT-01	Dashboard stats	<code>routes/reports.py::get_stats</code>	<code>AdminDashboard.js</code> , <code>Reports.js</code>	reservations
US-RPT-02	Date/building filter	<code>routes/reports.py::get_stats</code> (query params)	<code>Reports.js</code>	reservations
US-RPT-03	AI insights	<code>routes/reports.py::generate_ai_insights</code>	<code>Reports.js</code>	ai_insights
US-RPT-04	Insights history	<code>routes/reports.py::get_insights_history</code>	<code>Reports.js</code>	ai_insights
US-RPT-05	Admin reservations + cancellation with reason	<code>routes/admin.py::get_admin_reservations</code> , <code>admin_cancel_reservation</code> (requires reason, sends notification)	<code>AdminReservations.jsx</code>	reservations, notifications

1.10 Admin — Event Blocking

Req ID	Requirement	Backend Artifact	Frontend Artifact	DB Collection
US-EVT-01	Block slots for event	<code>routes/event_blocks.py::create_event_block</code>	<code>EventBlocking.jsx</code>	event_blocks, reservations
US-EVT-02	List event blocks	<code>routes/event_blocks.py::get_event_blocks</code>	<code>EventBlocking.jsx</code>	event_blocks
US-EVT-03	Delete event block	<code>routes/event_blocks.py::delete_event_block</code>	<code>EventBlocking.jsx</code>	event_blocks, reservations

1.12 Admin — Site Customization

Req ID	Requirement	Backend Artifact	Frontend Artifact	DB Collection	Test Coverage	Status
US-SITE-01	Login page editor	<code>routes/site_content.py</code>	<code>AdminSettings.js</code>	site_content	TC-SITE-01	Implemented

1.13 Admin — Building Policies

Req ID	Requirement	Backend Artifact	Frontend Artifact
US-POL-01	Create building policy	<code>routes/building_policies.py::create_building_policy</code>	<code>BuildingPolicies.js</code>
US-POL-02	Sticker requirement	<code>routes/reservations.py::create_reservation</code> (vehicle plate check)	<code>BookingPage.js</code>
US-POL-03	Slot registration	<code>routes/building_policies.py::create_slot_registration</code>	<code>BuildingPolicies.js</code>
US-POL-04	Open floors	<code>routes/reservations.py</code> (open_floor_ids bypass)	<code>BuildingPolicies.js</code>
US-POL-05	Zone-filtered user selection	<code>routes/building_policies.py</code> + frontend filtering	<code>BuildingPolicies.js</code>
US-POL-06	Vehicle dropdown	<code>routes/vehicles.py::get_vehicles</code>	<code>BuildingPolicies.js</code>

1.14 Parking Attendant

Req ID	Requirement	Backend Artifact	Frontend Artifact
US-ATT-01	Daily reservations	<code>routes/attendant.py::get_attendant_daily_reservations</code>	<code>AttendantDashboard.js</code>
US-ATT-02	Confirm arrival	<code>routes/reservations.py::confirm_reservation</code>	<code>AttendantDashboard.js</code>
US-ATT-03	Report no-show	<code>routes/attendant.py::report_no_show</code>	<code>AttendantDashboard.js</code>
US-ATT-04	Today button	—	<code>AttendantDashboard.js</code>
US-ATT-05	QR scan	<code>routes/reservations.py::scan_qr_lookup</code>	<code>ScanPage.js</code>
US-ATT-06	Summary cards	—	<code>AttendantDashboard.js</code>

2. Business Rules Compliance

Rule ID	Rule	Implementation	Enforcement Point	Verified
BR-01	Vehicle required for booking	<code>BookingPage.js</code> validates vehicle selection; <code>create_reservation</code> checks vehicle exists	Frontend + Backend	Yes
BR-02	No double-booking	Compound index <code>(slot_id, date, status)</code> . Backend checks existing reservations before insert	Backend (atomic)	Yes
BR-03	External booking: single-day + reason	<code>create_reservation</code> validates <code>is_external_booking</code> → enforces <code>len(dates) == 1</code> and <code>reason</code>	Backend	Yes
BR-04	Blocked users cannot access	<code>login</code> rejects blocked users; <code>create_reservation</code> checks <code>is_blocked</code>	Backend (auth + business)	Yes
BR-05	No-show only for today/past	<code>report_no_show</code> checks <code>reservation.date <= today</code> . Records <code>no_show_at</code> timestamp. Frontend hides buttons for future dates	Backend + Frontend	Yes
BR-06	30-min session expiry	<code>JWT_EXPIRATION_HOURS=0.5</code> in <code>.env</code> . Cookie <code>max-age=1800</code> . JWT <code>exp</code> claim validated	Backend (JWT)	Yes
BR-07	VIP/Group Head privileges	<code>tags</code> field on users. <code>create_reservation</code> skips single-day restriction for VIP	Backend	Yes
BR-08	Maintenance slots not bookable	<code>get_available_slots</code> excludes <code>status: maintenance</code> . Frontend marks as unavailable	Backend + Frontend	Yes
BR-09	Booking window enforcement	<code>parking_configs.booking_window_days</code> . Frontend limits calendar max date	Frontend + Config	Yes
BR-10	First-login password change	<code>must_change_password</code> flag. <code>AuthContext.js</code> shows <code>ForcePasswordChange</code> dialog	Backend + Frontend	Yes
BR-11	Waitlist notification expiry	<code>check_waitlist_expiry()</code> background task. <code>notified_at</code> + window minutes	Backend	Yes

Rule ID	Rule	Implementation	Enforcement Point	Verified
BR-12	No-show release based on no_show_at	<code>auto_mark_no_shows()</code> sets <code>no_show_at</code> . Slot release = <code>no_show_at</code> + configured minutes	Backend	Yes
BR-13	Plate number uniqueness	<code>create_vehicle</code> checks <code>vehicles</code> collection for existing plate before insert. Returns 400 if duplicate	Backend	Yes
BR-14	Vehicle-level sticker validation	<code>create_reservation</code> checks <code>slot_registrations</code> for matching <code>vehicle_plate</code> + <code>building_id</code> . User registration alone is insufficient	Backend	Yes
BR-15	Vehicle-to-slot registration for non-open floors	<code>create_reservation</code> checks <code>slot_registrations</code> for matching <code>slot_id</code> + <code>user_id</code> + <code>vehicle_plate</code> on non-open floors	Backend	Yes
BR-16	Admin cancellation requires reason; parker notified	<code>routes/admin.py::admin_cancel_reservation</code> requires <code>reason</code> body field; creates in-app notification for parker containing reason; triggers waitlist	Backend + Notifications	Yes
BR-17	Main building exclusivity (admins exempt)	<code>routes/reservations.py::create_reservation</code> checks <code>parking_configs.main_building_exclusive</code> ; compares <code>user.main_building</code> vs <code>building_id</code> ; admin role bypasses check	Backend	Yes
BR-18	Event blocks create confirmed reservations	<code>routes/event_blocks.py::create_event_block</code> creates one reservation per slot with <code>booking_type: "event_block"</code> , <code>status: "confirmed"</code> to prevent double-booking	Backend	Yes
BR-19	Self-service password change requires current password	<code>routes/auth.py::change_password</code> verifies <code>current_password</code> hash via bcrypt before updating. Returns 400 if incorrect	Backend	Yes

Rule ID	Rule	Implementation	Enforcement Point	Verified
BR-20	Forgot password notifies admins; prevents email enumeration	<code>routes/auth.py::forgot_password</code> looks up user, creates notification for all admins if found. Always returns 200 regardless of email existence	Backend	Yes

3. Non-Functional Requirements Compliance

NFR ID	Requirement	Target	Implementation	Verified
NFR-01	Mobile responsive	375px+	TailwindCSS responsive classes, mobile bottom nav	Yes
NFR-02	Page load < 3s	< 3 sec	React SPA, <code>Promise.allSettled</code> parallel fetching, retry logic	Yes
NFR-03	API response < 500ms	< 500ms (p95)	25 MongoDB indexes, Motor async driver, connection pooling (maxPool=50)	Yes
NFR-04	200+ concurrent users	200+	Stateless JWT, async I/O, DB connection pooling	Designed
NFR-05	Data security	Multi-layer	HttpOnly cookies, bcrypt, CSP/HSTS/X-Frame headers, rate limiting, device fingerprinting, RBAC. Password inputs use uncontrolled components (<code>useRef</code>) to prevent plaintext credential exposure in the DOM (CWE-312 mitigation)	Yes
NFR-06	99.9% uptime	99.9%	Container orchestration, health checks, graceful shutdown, supervisor auto-restart	Designed
NFR-07	Browser support	Latest 2 versions	React 19, ES2020 target, no browser-specific APIs	Yes

4. Security Controls Matrix

Control	Category	Implementation	Layer
Authentication	Access Control	JWT tokens with 30-min expiry	Backend
Session Management	Access Control	DB-backed sessions with device fingerprinting	Backend
Authorization (RBAC)	Access Control	<code>get_current_user</code> , <code>require_admin</code> , <code>require_attendant</code> guards	Backend
Password Hashing	Data Protection	bcrypt with auto-salting	Backend
HttpOnly Cookies	Transport Security	<code>httponly=True</code> , <code>secure=True</code> , <code>samesite=lax</code>	Backend
HTTPS Enforcement	Transport Security	HSTS header (<code>max-age=31536000</code> ; <code>includeSubDomains</code>)	Middleware
Content Security Policy	Injection Prevention	CSP header restricting script/style/img sources	Middleware
Clickjacking Protection	Injection Prevention	<code>X-Frame-Options: DENY</code>	Middleware
XSS Protection	Injection Prevention	<code>X-XSS-Protection: 1; mode=block</code>	Middleware
Rate Limiting	Abuse Prevention	5 requests/minute on login endpoint	Backend
Input Validation	Data Integrity	Pydantic models for all request bodies	Backend
File Upload Validation	Data Integrity	Extension whitelist, size limit (10MB), content-type check	Backend
Path Traversal Prevention	Data Integrity	Regex sanitization + <code>resolve().is_relative_to()</code> check	Backend
Permission Boundary	Privacy	Camera, microphone, geolocation disabled via Permissions-Policy	Middleware
Blocked Account Enforcement	Authentication	Login endpoint checks <code>is_blocked</code> flag before issuing session token. Blocked users receive 403 regardless of credential validity	Backend
DOM Password Exposure	Data Protection	All password inputs use uncontrolled React components (<code>useRef</code>); password values are	Frontend

Control	Category	Implementation	Layer
Prevention (CWE-312)		never stored in component state or accessible in the DOM	
Attendant Building Validation	Data Integrity	Attendants must have at least one assigned building; backend validation prevents creation without building	Backend
One Slot Per User Per Building	Data Integrity	Users can only be registered to one slot per building for policy-based buildings	Backend
Hour Selection Restrictions	Business Logic	Past hours and hours outside parking config are disabled on booking page	Frontend
Vehicle Plate Length Validation	Data Integrity	Plate numbers must be 2-15 characters (Pydantic validators)	Backend

Previous: [ERD](#) | Back to: [System Design](#) | Next: [Test Plan](#)